

Building Cross-Platform Applications with .NET Core

“Write once, run anywhere” seems to be the mantra that finds favour with application developers nowadays. This reduces the need for developers to write a lot of redundant code. .NET Core, an open source offering from Microsoft, (yes, Microsoft!) is just the tool for writing code for a cross-platform application that will work on Windows, Linux and macOS systems.

.NET Core is a free and open source framework for developing cross-platform applications targeting Windows, Linux and macOS. It is capable of running applications on devices, the cloud and the IoT. It supports four cross-platform scenarios: ASP.NET Core Web apps, command line apps, libraries and Web APIs. The recently released .NET Core 3 (preview bits) supports Windows rendering forms like WinForms, WPF and UWP, but only on Windows.

Historically, the .NET Framework has worked only on the Windows platform. It was released in 2002, and has millions of applications developed using it, which run on Windows servers and desktops. The stable version of the .NET Framework (Windows only) release is 4.7.2. With the help of the Mono project, developers were able to bring the .NET code on to mobile devices, macOS, and Linux, but not the applications. So they were managing two separate applications for cross-platform compatibility. Now, with the help of .NET Core, they can target any platform without any code changes; in short, they can now “write once, and run anywhere.”

.NET Core is an open source development platform and Microsoft contributed it to the .NET Foundation in 2014. It is now one of the most active .NET Foundation projects. It can be freely adopted by individuals and companies for personal, academic or commercial purposes. Multiple companies use .NET Core as a part of apps, tools, new platforms and hosting services. Some of these companies make significant contributions to .NET Core on GitHub, and provide guidance on product direction as part of the .NET Foundation Technical Steering Group. This group of companies includes Microsoft, Google, Red Hat, Unity, Samsung and JetBrains.

With .NET Core, developers can build and run .NET code on more than one platform of choice. As of this November, .NET Core 2.1 is generally available (production ready)

whereas .NET Core 3.0 is under development, having had a few preview releases. .NET 3.0 GA is expected by early 2019.

Why should developers care about .NET Core?

Until 2010, every organisation wanted to design its applications using the SOA (service oriented architecture) pattern with API interfaces. The server operating system was chosen first and then the client components. But once the server platform is confirmed, it's hard to switch between the platforms. Java is well positioned and has become a natural choice for creating cross-platform apps, with Node, Python, etc, now added to this list.

Key things to consider about .NET Core

1. **Cross-platform:** It runs on Windows, macOS and Linux operating systems. With .NET Core you can target any application type that's running on any platform. Developers can reuse skills and code across all of them in a familiar environment. From mobile applications running on iOS, Android and Windows, to enterprise server applications running on Windows Server and Linux, or high scale microservices running in the cloud, .NET Core provides a solution for you.
2. **Consistent across architectures:** It runs your code with the same behaviour on multiple architectures, including x64, x86 and ARM.
3. **Flexible deployment:** .NET Core can be included in your app or installed side-by-side, user-wide or machine-wide. It can be used with Docker containers, which typically run Linux today, and it can host ASP.NET Core applications, allowing them to take advantage of the benefits of containers and microservices.
4. **Modular:** .NET Core supports the use of NuGet packages. Instead of assemblies, now developers can work with NuGet packages. In the .NET Framework, the framework updates are typically serviced through Windows updates, but .NET